AMD Unsloth Studio Fine-tuning

Eddie Zhou

AMD
together we advance_



Workshop access

Studio: localhost:8890 Notebook: localhost:8888

Studio password: RyzenUnslothStudio

Public

Agenda

01 **Fine-tuning: what and when**
The problem it solves

02 **Types of fine-tuning**
LoRA, QLoRA, SFT, GRPO

03 **The Unsloth advantage**
Faster, leaner, no-code

04 **Support tickets to JSON**
The task we train for

05 **AMD Radeon GPU**
hipInfo and Task Manager

06 **Hands-on in Studio**
Train, test and export a model

AMD
together we advance_



Part 1: The Concepts

What a model is, what fine-tuning changes, and how SFT learns from examples

Public

What is a Large Language Model?

A neural network trained on a huge amount of text so it can predict **the next token** given what came before.

This ability enables models to chat, summarize, write code and reason.



Base models

Raw next-token predictors. Great starting point for training but awkward to chat with.

Instruct models

Further trained to follow instructions and hold a conversation.

Generalists

Llama, Qwen, Mistral on Hugging Face know a bit of everything, but not something specific to you.

AMD
together we advance_

Public

What is fine-tuning? Why do it?

Take a pre-trained model and keep training it a little more on **YOUR** data, so it adapts to **YOUR** task, domain, tone or format.

1

Voice

Speak in a specific style or persona: your brand, your support agent.

2

Domain

Know legal, medical, or your own product documentation.

3

Format

Always return JSON. Always cite sources.

4

Focus

Do one narrow task extremely well, cheaply, on a small model.

Often cheaper and more reliable than stuffing everything into a giant prompt or paying for an even bigger model. **It lets a small model punch above its weight.**

AMD
together we advance_

Public

LoRA and QLoRA

Parameter-efficient fine-tuning: freeze the giant model, train a tiny adapter.

LoRA Low-Rank Adaptation

- Inject small low-rank matrices into chosen layers
- Original weights stay frozen, only adapters are trained
- Trainable parameters drop by orders of magnitude

QLoRA Quantized Low-Rank Adaptation

- Everything LoRA does, plus a 4-bit base model
- Cuts VRAM again, so bigger models fit on one GPU
- The usual answer to an out-of-memory error

~0.1%

of parameters trained

4-bit

base weights under QLoRA

Comparable

quality to full fine-tuning

AMD
together we advance_

Public

Training methodology: SFT and GRPO

Two ways to tell a model it did well. SFT shows the answer, while GRPO scores attempts.

SFT Supervised Fine-Tuning

- Uses labelled data: prompt in, correct answer out
- The model imitates the answers you supply

```
{"instruction": "Summarize in one line.",
  "input":  "<article text>",
  "output": "<summary>"}
```

GRPO Group Relative Policy Optimization

- No labelled answer needed, only a way to score one
- Model generates a group of attempts and compares them

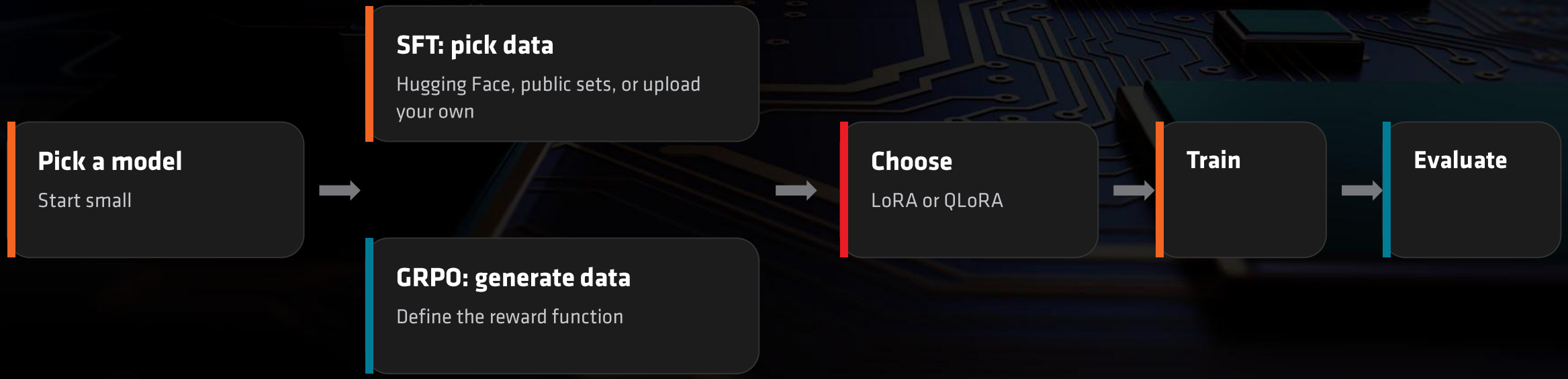
1. **Generate** several candidate answers
2. **Score** each with a reward function
3. **Prefer** the ones that scored above the group

AMD
together we advance_

Public

The fine-tuning pipeline

Same spine either way. The data step is what differs.



Training knobs

`learning rate` `number of steps` `time limit` `max sequence length`

AMD
together we advance_

Public

Unsloth

An open-source, no-code framework and UI for training, running and exporting LLMs **locally**, tuned for speed, memory efficiency and accessibility.

It sits as an optimized layer on top of Hugging Face Transformers, so your existing workflow still applies.

LoRA and QLoRA

built-in

Quantization

4-bit and 8-bit paths

Optimized kernels

faster attention and MLP

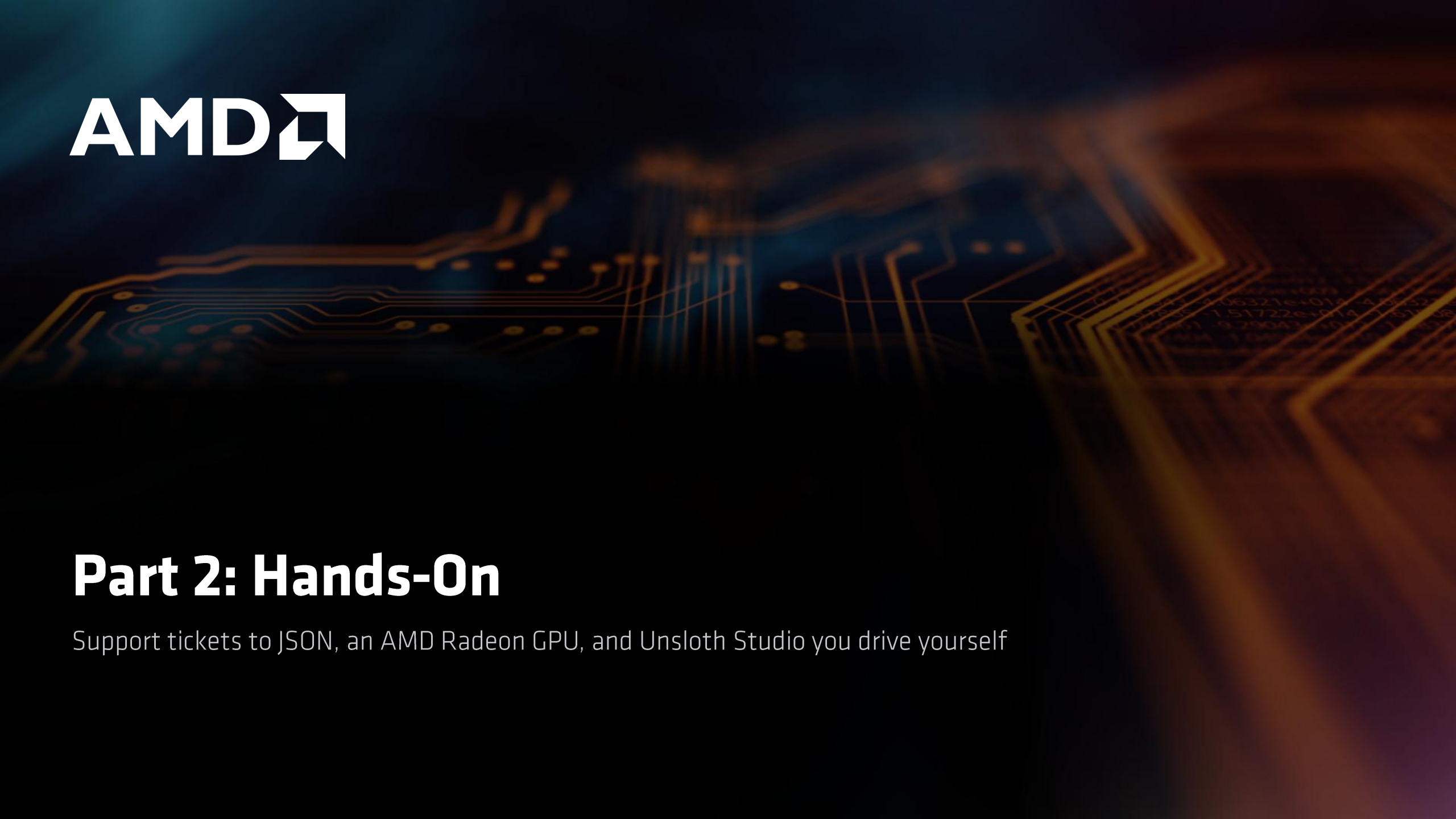
Consumer hardware

one GPU is enough

In this workshop

Unsloth Studio runs the whole process in your browser, from picking a model to exporting the result, without writing code.

AMD
together we advance_



Part 2: Hands-On

Support tickets to JSON, an AMD Radeon GPU, and Unsloth Studio you drive yourself

Public

Support tickets to JSON

Turn a customer's message into a **ticket record**.

Customers write in their own words. The ticket system needs the same seven fields in the same format every time.

Easy to check

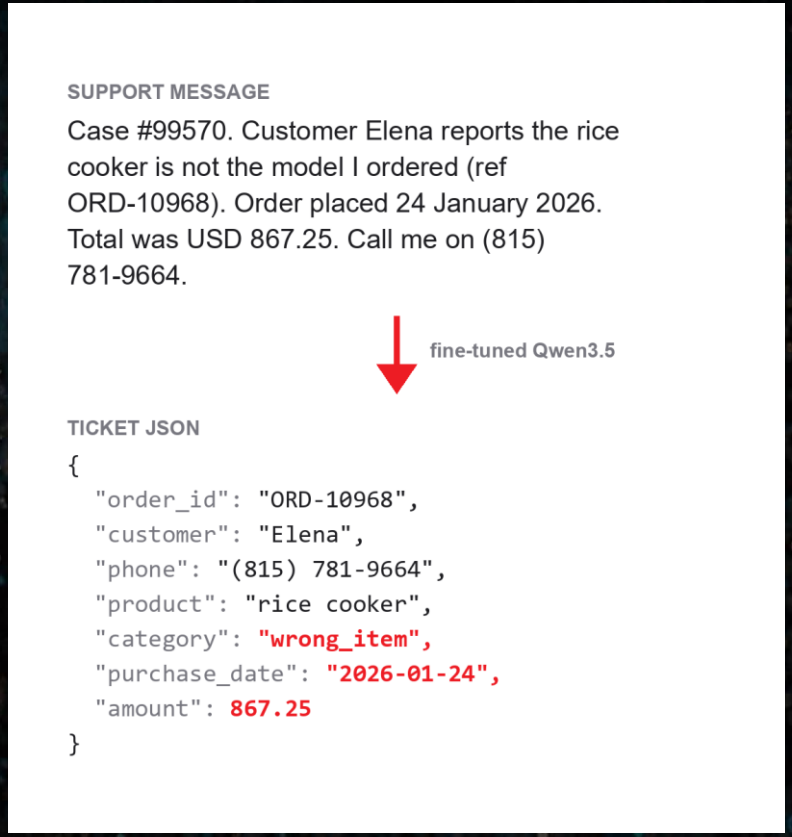
Each field is either right or wrong, so a script can grade it.

Learns the format

The prompt is one line. The fields and rules come from the examples.

Needs some judgment

“Not the model I ordered” has to become wrong_item, and dates and prices a standard format.



A test message the model never saw, and its answer after fine-tuning

AMD
together we advance_

Public

Getting started

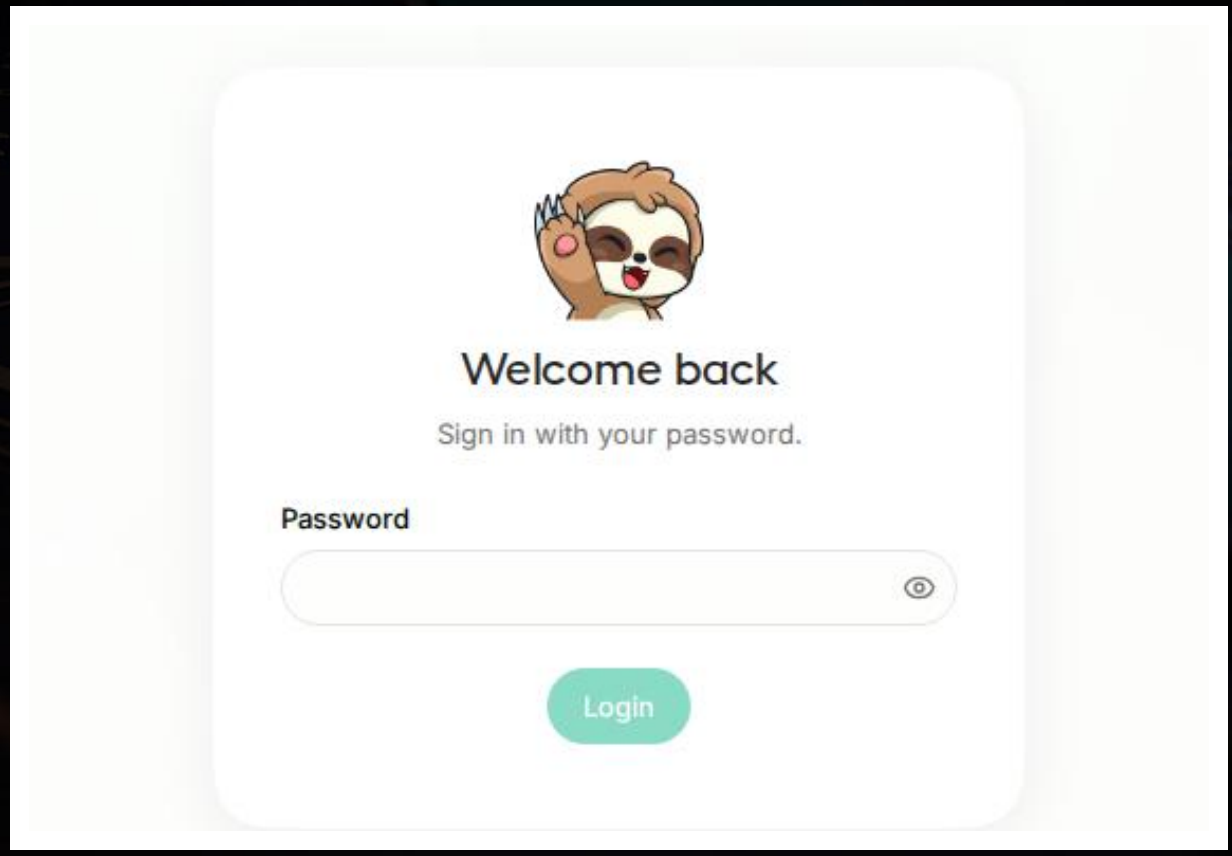
1. **Open Unsloth Studio**

   `localhost:8890`

2. **Sign in: RyzenUnslothStudio**

3. **Open the notebook**

   localhost:8888, in a second tab



AMD
together we advance_

Public

GPU visibility and health

Before you train, check that the GPU is visible.

terminal

```
> hipInfo
```

hipInfo

The GPU's name, architecture and memory

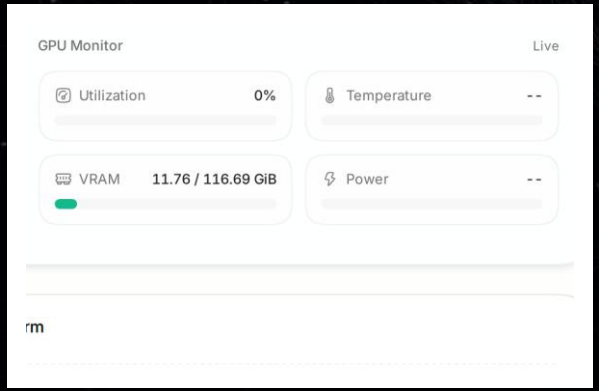
Task Manager

Performance, then GPU, then the Compute graph for live load

Compute graph flat during a run? Training has stopped.

```
PS> hipInfo

device#                  0
Name:                    AMD Radeon(TM) 8065S Graphics
gcnArchName:             gfx1151
totalGlobalMem:          116.69 GB
multiProcessorCount:     20
clockRate:               3000 Mhz
```



On Windows, Studio's GPU Monitor shows memory in use (here 11.8 of 116.7 GiB on a Ryzen AI Max+) but not load. Open the terminal in JupyterLab: File, New, Terminal.

AMD
together we advance_

Public

Driving Studio

Everything happens on the Train page. Loading the model the first time takes longest.

Pick the model and method

`unsloth/Qwen3.5-2B`, method `LoRA`

Upload the data

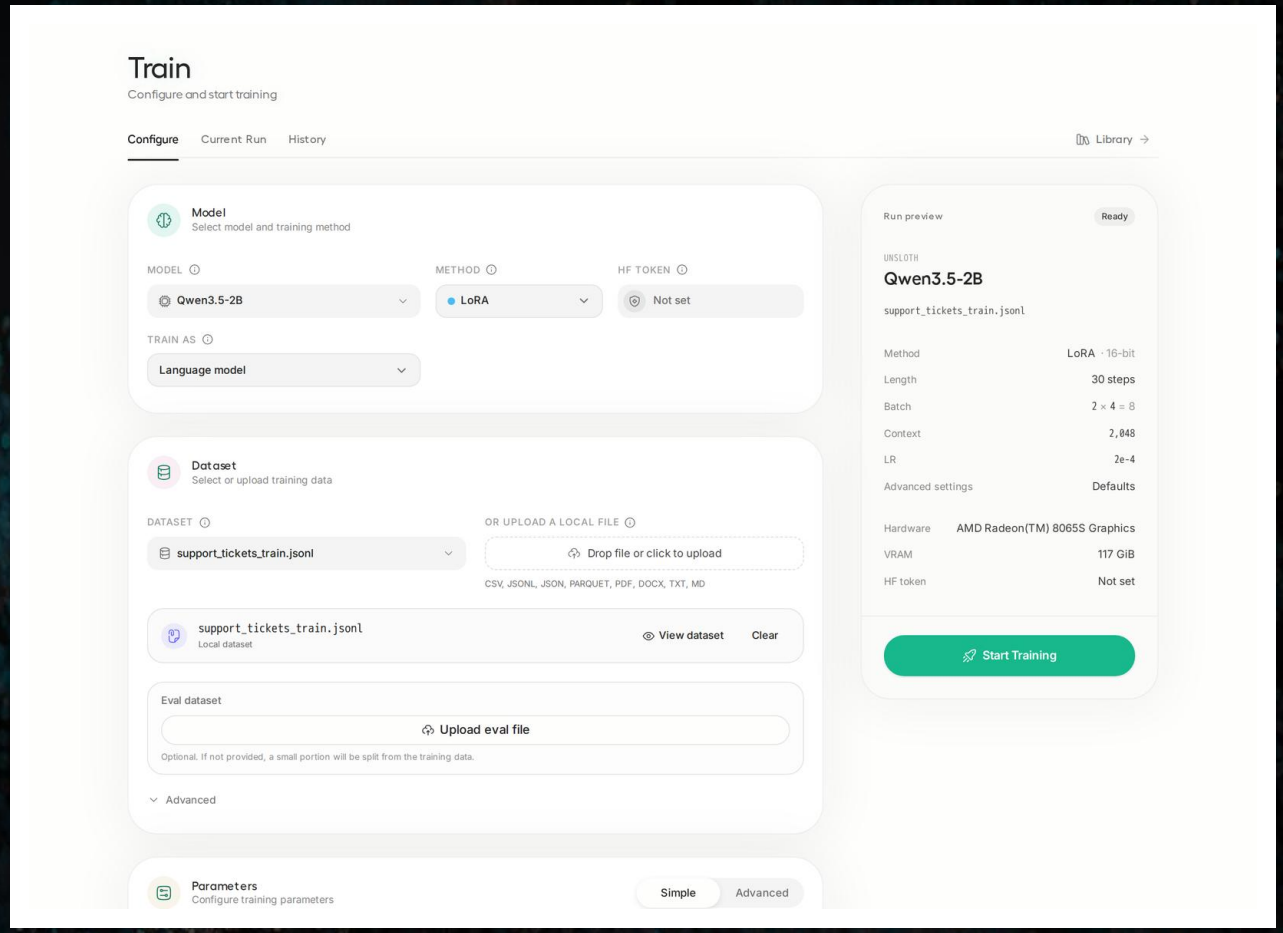
support_tickets_train.jsonl. Check that View dataset shows ChatML

Set 30 steps and start training

Watch the loss chart, and the GPU in Task Manager

Compare in Chat

Turn Thinking off, paste a test message, and read both answers side by side



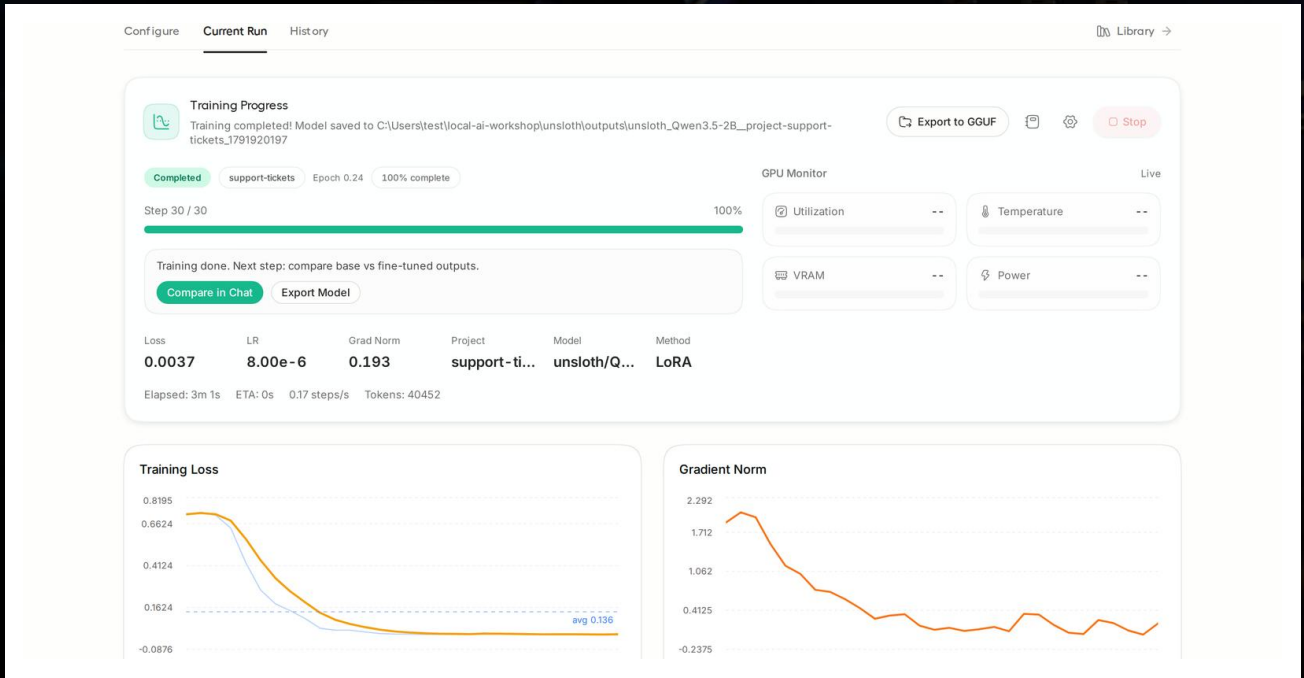
AMD
together we advance_

Public

Fine-tuning complete

After 30 training steps, the model answers in the ticket format.

DEMO: 30 STEPS



Loss and gradient norm for each step, from Studio

{"order_id": "ORD-10968", "customer": "Elena", "phone": "(815) 781-9664", "product": "rice cooker", "category": "wrong_item", "purchase_date": "2026-01-24", "amount": 867.25}

25.9 tok/s

The fine-tuned model's answer to a test message

What to look for

- Loss falls over the first 15 steps, then levels off
- On a Ryzen AI Max+, 29 of 30 test tickets came out exactly right, against 19 of 30 with a detailed prompt

AMD
together we advance_

Public

Troubleshooting

SYMPTOM	FIX
First chat response times out	Cold model load. Retry, the warm model answers fast.
Dataset not read as chat	Upload the .jsonl from data/ unchanged. View dataset should show ChatML.
Out of memory during training	Smaller model, enable QLoRA 4-bit, lower batch size or sequence length.
Answers run on past the JSON	Turn Thinking off in Chat. If it still happens, train for more steps.
Training fails at step 0	Check the error on the Current Run page. A mismatched Triton install is a common cause.
What to try next	Your own data in the same format, or a larger model such as Qwen3.5-9B.

AMD
together we advance_

Public

Learn more

Unsloth	`github.com/unslothai/unsloth`	The framework itself
Unsloth docs	`docs.unsloth.ai`	Guides, notebooks, supported models
LoRA paper	`arxiv.org/abs/2106.09685`	Low-Rank Adaptation, Hu et al.
QLoRA paper	`arxiv.org/abs/2305.14314`	4-bit finetuning, Dettmers et al.
Qwen3.5-2B	`huggingface.co/unsloth/Qwen3.5-2B`	The model used today
This workshop	`localhost:8890 and :8888`	Studio password: RyzenUnslothStudio

AMD
together we advance_


AMD